

Proyecto 3 - Ruta C: Pipeline Python, SQL Server Y Power BI

Profesor: M. Alan Badillo Salas

Este proyecto usa el archivo `financiera.csv` para simular un flujo empresarial de integración de datos.

La intención completa del proyecto sería automatizar un pipeline semanal:

```
CSV semanal
  -> Python ETL
      -> SQL Server Express 2022
          -> Power BI
```

Sin embargo, para una práctica de 1 hora se trabajará únicamente un punto concreto:

Leer el CSV con Python, generar un concentrado de riesgo por sucursal, guardarlo en SQL Server Express 2022 y recuperarlo desde Power BI.

1. Contexto Del Proyecto

En una empresa real, Power BI no siempre consume archivos CSV directamente.

Un flujo más empresarial suele ser:

1. El área operativa genera archivos CSV.
2. Python limpia y transforma los datos.
3. SQL Server almacena tablas listas para análisis.
4. Power BI se conecta a SQL Server.

Esto separa responsabilidades:

- Python procesa.

- SQL Server conserva.
- Power BI visualiza.

2. Pregunta De Negocio

La pregunta general del proyecto es:

¿Cómo crear una fuente confiable para analizar riesgo financiero en Power BI?

Para esta sesión se reducirá a:

¿Cómo generar una tabla resumida de riesgo por sucursal usando Python y consultarla desde Power BI mediante SQL Server?

3. Resultado Esperado En 1 Hora

Al terminar, el estudiante deberá tener:

- Un script de Python que lea `financiera.csv`.
 - Un concentrado por sucursal con montos esperados, recibidos y faltantes.
 - Una columna sencilla de clasificación de riesgo.
 - Una base de datos SQL Server llamada `FinancieraRiesgo`.
 - Una tabla SQL llamada `dbo.resumen_riesgo_sucursal`.
 - Power BI conectado a SQL Server Express 2022.
-

4. Teoría Mínima Necesaria

Un proceso ETL significa:

```
Extract    -> Extraer datos desde una fuente.  
Transform  -> Limpiar, calcular y resumir.  
Load       -> Cargar el resultado en un destino.
```

En este proyecto:

Fase	Herramienta	Acción
Extract	Python	Leer <code>financiera.csv</code>
Transform	Python	Agrupar por sucursal y calcular riesgo
Load	Python + SQL Server	Guardar una tabla analítica
Visualizar	Power BI	Conectarse a SQL Server

La idea importante es:

No mandar a Power BI el CSV crudo, sino una tabla ya preparada para análisis.

5. Requisitos Del Laboratorio

Para esta práctica se usará específicamente:

- SQL Server Express 2022.
- Power BI Desktop.
- Python 3.
- Librería `pandas`.
- Librería `pyodbc`.
- ODBC Driver 17 o 18 for SQL Server.

Instalación de librerías de Python:

```
pip install pandas pyodbc
```

Servidor esperado para SQL Server Express:

```
localhost\SQLEXPRESS
```

Si el equipo usa otro nombre de instancia, reemplazarlo en el script.

Ejemplos comunes:

```
.\SQLEXPRESS
localhost\SQLEXPRESS
NOMBRE_DEL_EQUIPO\SQLEXPRESS
```

6. Actividad Guiada: CSV A SQL Server Con Python

Paso 1. Crear El Archivo Python

Crear un archivo llamado:

```
pipeline_riesgo_sqlserver.py
```

Debe estar en la misma carpeta que:

```
financiera.csv
```

Paso 2. Leer El CSV

Contenido inicial:

```
from datetime import date

import pandas as pd

csv_path = "financiera.csv"

df = pd.read_csv(csv_path)

print(df.head())
print(df.columns)
```

Ejecutar:

```
python pipeline_riesgo_sqlserver.py
```

El objetivo de este paso es confirmar que Python puede abrir el archivo.

Paso 3. Convertir Tipos De Datos

Agregar debajo de la lectura:

```
df["pagoFechaEsperada"] = pd.to_datetime(df["pagoFechaEsperada"], errors="c

columnas_monto = [
    "pagoMontoEsperado",
    "pagoMontoRecibido",
    "pagoMontoFaltante",
]

for columna in columnas_monto:
    df[columna] = pd.to_numeric(df[columna], errors="coerce").fillna(0)
```

Esto hace dos cosas:

- Convierte fechas inválidas en valores nulos.
- Convierte montos a números.

Paso 4. Crear Una Variable De Vencimiento

Usaremos una fecha de corte fija para que todos obtengan el mismo resultado:

```
fecha_corte = pd.Timestamp(date(2026, 5, 18))

df["dias_vencidos"] = (fecha_corte - df["pagoFechaEsperada"]).dt.days
df["es_vencido"] = (df["dias_vencidos"] > 0) & (df["pagoMontoFaltante"] > 0)
df["monto_faltante_vencido"] = df["pagoMontoFaltante"].where(df["es_vencido"]
```

Interpretación:

Un pago está vencido si su fecha esperada ya pasó
y todavía tiene monto faltante.

Paso 5. Crear Un Concentrado Por Sucursal

Agregar:

```
resumen = (
    df.groupby("sucursalOrigenFolio", dropna=False)
    .agg(
        pagos=("amortizacionId", "count"),
        monto_esperado=("pagoMontoEsperado", "sum"),
        monto_recibido=("pagoMontoRecibido", "sum"),
        monto_faltante=("pagoMontoFaltante", "sum"),
        monto_faltante_vencido=("monto_faltante_vencido", "sum"),
        pagos_vencidos=("es_vencido", "sum"),
    )
    .reset_index()
)

resumen["sucursalOrigenFolio"] = resumen["sucursalOrigenFolio"].fillna("SIN")
resumen["porcentaje_mora"] = resumen["monto_faltante_vencido"] / resumen["monto_faltante"]
resumen["porcentaje_mora"] = resumen["porcentaje_mora"].fillna(0)
resumen["porcentaje_mora"] = resumen["porcentaje_mora"].replace([float("inf")], 0)
```

Este concentrado produce una fila por sucursal.

Nota: `monto_faltante` incluye cualquier monto pendiente, incluso pagos futuros. Por eso el riesgo se calcula con `monto_faltante_vencido`, que solo considera pagos atrasados a la fecha de corte.

Paso 6. Clasificar Riesgo

Agregar:

```
def clasificar_riesgo(porcentaje_mora):
    if porcentaje_mora >= 0.40:
        return "Critico"
    if porcentaje_mora >= 0.20:
        return "Alto"
    if porcentaje_mora >= 0.10:
        return "Regular"
    if porcentaje_mora > 0:
        return "Bajo"
    return "Sin mora"

resumen["riesgo"] = resumen["porcentaje_mora"].apply(clasificar_riesgo)
resumen["fecha_corte"] = fecha_corte.date()

print(resumen.head())
```

La clasificación es sencilla, pero suficiente para generar una tabla útil para Power BI.

Paso 7. Crear La Base Y Tabla En SQL Server

El script se conectará primero a la base `master` para crear la base `FinancieraRiesgo` si no existe.

Después se conectará a `FinancieraRiesgo`, recreará la tabla `dbo.resumen_riesgo_sucursal` e insertará el concentrado.

Agregar al final:

```
import pyodbc

servidor = r"localhost\SQLEXPRESS"
base_datos = "FinancieraRiesgo"
driver = "ODBC Driver 18 for SQL Server"

conexion_master = pyodbc.connect(
    f"DRIVER={{driver}};"
    f"SERVER={servidor};"
    "DATABASE=master;"
    "Trusted_Connection=yes;"
    "TrustServerCertificate=yes;",
```

```

        autocommit=True,
    )

    cursor_master = conexion_master.cursor()
    cursor_master.execute(f"""
    IF DB_ID('{base_datos}') IS NULL
    BEGIN
        CREATE DATABASE {base_datos};
    END
    """)
    cursor_master.close()
    conexion_master.close()

    conexion = pyodbc.connect(
        f"DRIVER={{driver}};"
        f"SERVER={servidor};"
        f"DATABASE={base_datos};"
        "Trusted_Connection=yes;"
        "TrustServerCertificate=yes;",
    )

    cursor = conexion.cursor()

    cursor.execute("""
    IF OBJECT_ID('dbo.resumen_riesgo_sucursal', 'U') IS NOT NULL
    BEGIN
        DROP TABLE dbo.resumen_riesgo_sucursal;
    END;

    CREATE TABLE dbo.resumen_riesgo_sucursal (
        sucursalOrigenFolio NVARCHAR(50) NOT NULL,
        pagos INT NOT NULL,
        monto_esperado DECIMAL(18, 2) NOT NULL,
        monto_recibido DECIMAL(18, 2) NOT NULL,
        monto_faltante DECIMAL(18, 2) NOT NULL,
        monto_faltante_vencido DECIMAL(18, 2) NOT NULL,
        pagos_vencidos INT NOT NULL,
        porcentaje_mora DECIMAL(18, 6) NOT NULL,
        riesgo NVARCHAR(20) NOT NULL,
        fecha_corte DATE NOT NULL
    );
    """)

    insert_sql = """
    INSERT INTO dbo.resumen_riesgo_sucursal (

```



```

        sucursalOrigenFolio,
        pagos,
        monto_esperado,
        monto_recibido,
        monto_faltante,
        monto_faltante_vencido,
        pagos_vencidos,
        porcentaje_mora,
        riesgo,
        fecha_corte
    )
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?);
"""

filas = [
    (
        fila.sucursalOrigenFolio,
        int(fila.pagos),
        float(fila.monto_esperado),
        float(fila.monto_recibido),
        float(fila.monto_faltante),
        float(fila.monto_faltante_vencido),
        int(fila.pagos_vencidos),
        float(fila.porcentaje_mora),
        fila.riesgo,
        fila.fecha_corte,
    )
    for fila in resumen.itertuples(index=False)
]

cursor.fast_executemany = True
cursor.executemany(insert_sql, filas)
conexion.commit()

cursor.close()
conexion.close()

print("Base FinancieraRiesgo y tabla dbo.resumen_riesgo_sucursal creadas co

```

Si el equipo tiene instalado el driver 17 en lugar del 18, cambiar esta línea:

```
driver = "ODBC Driver 18 for SQL Server"
```

por:

```
driver = "ODBC Driver 17 for SQL Server"
```

7. Script Completo

```
from datetime import date

import pandas as pd
import pyodbc

csv_path = "financiera.csv"

df = pd.read_csv(csv_path)

df["pagoFechaEsperada"] = pd.to_datetime(df["pagoFechaEsperada"], errors="c

columnas_monto = [
    "pagoMontoEsperado",
    "pagoMontoRecibido",
    "pagoMontoFaltante",
]

for columna in columnas_monto:
    df[columna] = pd.to_numeric(df[columna], errors="coerce").fillna(0)

fecha_corte = pd.Timestamp(date(2026, 5, 18))

df["dias_vencidos"] = (fecha_corte - df["pagoFechaEsperada"]).dt.days
df["es_vencido"] = (df["dias_vencidos"] > 0) & (df["pagoMontoFaltante"] > 0
df["monto_faltante_vencido"] = df["pagoMontoFaltante"].where(df["es_vencido

resumen = (
    df.groupby("sucursalOrigenFolio", dropna=False)
    .agg(
        pagos=("amortizacionId", "count"),
        monto_esperado=("pagoMontoEsperado", "sum"),
        monto_recibido=("pagoMontoRecibido", "sum"),
        monto_faltante=("pagoMontoFaltante", "sum"),
        monto_faltante_vencido=("monto_faltante_vencido", "sum"),
        pagos_vencidos=("es_vencido", "sum"),
```

```

    )
    .reset_index()
)

resumen["sucursalOrigenFolio"] = resumen["sucursalOrigenFolio"].fillna("SIN")
resumen["porcentaje_mora"] = resumen["monto_faltante_vencido"] / resumen["monto"]
resumen["porcentaje_mora"] = resumen["porcentaje_mora"].fillna(0)
resumen["porcentaje_mora"] = resumen["porcentaje_mora"].replace([float("inf"), float("-inf")], 0)

def clasificar_riesgo(porcentaje_mora):
    if porcentaje_mora >= 0.40:
        return "Critico"
    if porcentaje_mora >= 0.20:
        return "Alto"
    if porcentaje_mora >= 0.10:
        return "Regular"
    if porcentaje_mora > 0:
        return "Bajo"
    return "Sin mora"

resumen["riesgo"] = resumen["porcentaje_mora"].apply(clasificar_riesgo)
resumen["fecha_corte"] = fecha_corte.date()

servidor = r"localhost\SQLEXPRESS"
base_datos = "FinancieraRiesgo"
driver = "ODBC Driver 18 for SQL Server"

conexion_master = pyodbc.connect(
    f"DRIVER={{driver}};"
    f"SERVER={servidor};"
    "DATABASE=master;"
    "Trusted_Connection=yes;"
    "TrustServerCertificate=yes;",
    autocommit=True,
)

cursor_master = conexion_master.cursor()
cursor_master.execute(f"""
IF DB_ID('{base_datos}') IS NULL
BEGIN
    CREATE DATABASE {base_datos};
END
""")

```

```

cursor_master.close()
conexion_master.close()

conexion = pyodbc.connect(
    f"DRIVER={{driver}};"
    f"SERVER={servidor};"
    f"DATABASE={base_datos};"
    "Trusted_Connection=yes;"
    "TrustServerCertificate=yes;",
)

cursor = conexion.cursor()

cursor.execute("""
IF OBJECT_ID('dbo.resumen_riesgo_sucursal', 'U') IS NOT NULL
BEGIN
    DROP TABLE dbo.resumen_riesgo_sucursal;
END;

CREATE TABLE dbo.resumen_riesgo_sucursal (
    sucursalOrigenFolio NVARCHAR(50) NOT NULL,
    pagos INT NOT NULL,
    monto_esperado DECIMAL(18, 2) NOT NULL,
    monto_recibido DECIMAL(18, 2) NOT NULL,
    monto_faltante DECIMAL(18, 2) NOT NULL,
    monto_faltante_vencido DECIMAL(18, 2) NOT NULL,
    pagos_vencidos INT NOT NULL,
    porcentaje_mora DECIMAL(18, 6) NOT NULL,
    riesgo NVARCHAR(20) NOT NULL,
    fecha_corte DATE NOT NULL
);
""")

insert_sql = """
INSERT INTO dbo.resumen_riesgo_sucursal (
    sucursalOrigenFolio,
    pagos,
    monto_esperado,
    monto_recibido,
    monto_faltante,
    monto_faltante_vencido,
    pagos_vencidos,
    porcentaje_mora,
    riesgo,
    fecha_corte

```

```

)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?);
"""

filas = [
    (
        fila.sucursalOrigenFolio,
        int(fila.pagos),
        float(fila.monto_esperado),
        float(fila.monto_recibido),
        float(fila.monto_faltante),
        float(fila.monto_faltante_vencido),
        int(fila.pagos_vencidos),
        float(fila.porcentaje_mora),
        fila.riesgo,
        fila.fecha_corte,
    )
    for fila in resumen.itertuples(index=False)
]

cursor.fast_executemany = True
cursor.executemany(insert_sql, filas)
conexion.commit()

cursor.close()
conexion.close()

print(resumen.head())
print("Base FinancieraRiesgo y tabla dbo.resumen_riesgo_sucursal creadas co

```

Ejecutar:

```
python pipeline_riesgo_sqlserver.py
```

8. Validar La Tabla En SQL Server

Abrir SQL Server Management Studio o Azure Data Studio.

Conectarse al servidor:

```
localhost\SQLEXPRESS
```

Autenticación:

Windows Authentication

Abrir una nueva consulta sobre la base:

FinancieraRiesgo

Ejecutar:

```
SELECT TOP 10
    sucursalOrigenFolio,
    pagos,
    monto_esperado,
    monto_recibido,
    monto_faltante_vencido,
    porcentaje_mora,
    riesgo,
    fecha_corte
FROM dbo.resumen_riesgo_sucursal
ORDER BY monto_faltante_vencido DESC;
```

Si aparecen filas, el pipeline Python -> SQL Server funcionó correctamente.

9. Recuperar La Tabla En Power BI Desde SQL Server

Paso 1. Abrir El Conector

1. Abrir Power BI Desktop.
2. Seleccionar **Obtener datos**.
3. Elegir **SQL Server**.
4. Presionar **Conectar**.

Paso 2. Configurar La Conexión

En la ventana de SQL Server:

Servidor:

localhost\SQLEXPRESS

Base de datos:

FinancieraRiesgo

Modo de conectividad:

Importar

Para esta práctica se recomienda **Importar**, porque es más simple y suficiente para una tabla resumida.

Presionar **Aceptar**.

Paso 3. Elegir Autenticación

Seleccionar:

Windows

Después presionar:

Conectar

Si Power BI pregunta por el nivel de privacidad, elegir:

Organizacional

o, para laboratorio local:

Ninguno

Paso 4. Seleccionar La Tabla

En el navegador:

1. Expandir la base `FinancieraRiesgo` .
2. Seleccionar:

```
dbo.resumen_riesgo_sucursal
```

1. Revisar la vista previa.
2. Presionar **Cargar**.

Paso 5. Alternativa Con Consulta SQL

Si se prefiere traer solo las columnas necesarias:

1. En la ventana de conexión de SQL Server, abrir **Opciones avanzadas**.
2. En **Instrucción SQL**, escribir:

```
SELECT
    sucursalOrigenFolio,
    pagos,
    monto_esperado,
    monto_recibido,
    monto_faltante_vencido,
    pagos_vencidos,
    porcentaje_mora,
    riesgo,
    fecha_corte
FROM dbo.resumen_riesgo_sucursal;
```

1. Presionar **Aceptar**.

10. Visual Mínimo En Power BI

Crear una página con:

1. Tarjeta:


```
Suma de monto_faltante_vencido
```

1. Barras:

```
Eje: sucursalOrigenFolio  
Valores: monto_faltante_vencido
```

1. Tabla:

```
sucursalOrigenFolio  
monto_esperado  
monto_recibido  
monto_faltante_vencido  
pagos_vencidos  
porcentaje_mora  
riesgo  
fecha_corte
```

1. Segmentador:

```
riesgo
```

11. Entregable

El estudiante debe entregar:

- Script `pipeline_riesgo_sqlserver.py`.
- Base de datos `FinancieraRiesgo` en SQL Server Express 2022.
- Tabla `dbo.resumen_riesgo_sucursal`.
- Captura de Power BI conectado a SQL Server.
- Visual con riesgo por sucursal.

12. Aplicaciones Futuras

Este ejercicio crea una sola tabla resumida, pero puede crecer hacia un pipeline más realista.

Aplicaciones futuras:

- Crear una tabla `staging_pagos` con los datos crudos del CSV.
- Crear una tabla `fact_pagos` con datos limpios y tipados.
- Guardar snapshots semanales de riesgo para analizar evolución histórica.
- Agregar una tabla `dim_riesgo` para administrar las categorías de riesgo.
- Crear un procedimiento almacenado `sp_procesar_pagos`.
- Automatizar el script con Task Scheduler.
- Generar alertas para sucursales con mora alta.
- Conectar Power BI Service usando gateway local para actualizaciones programadas.

13. Conclusiones

Este ejercicio muestra una ruta cercana al trabajo real de datos:

Power BI no siempre debe hacer toda la limpieza. Muchas veces consume datos ya preparados por un proceso ETL.

El trabajo realizado permitió leer un CSV con Python, convertir tipos de datos, calcular variables de vencimiento, crear un resumen por sucursal y guardarlo en SQL Server Express 2022.

La conclusión principal es que SQL Server funciona como una capa intermedia confiable entre los archivos operativos y los reportes de Power BI. Esto permite separar el procesamiento, conservar resultados y construir reportes más consistentes.